

Understanding CODI's Low-Rank Answer Readout

A first-principles teaching guide to the architecture, the 28-direction discovery, the causal experiment, the rank-32 algebra, and what the speed measurements really prove.

The destination: by the end, you should be able to derive the low-rank readout on a whiteboard, explain where every matrix comes from, and defend the difference between an 11.3x component speedup and an estimated 15% end-to-end latency saving.

Experimental object	Verified setting
Model	Official zen-E/CODI-gpt2 checkpoint
Task	GSM8K, 1,319 evaluation questions
Representation	768-dimensional state at the answer-cue colon
Scientific result	PC 4-31: 28 directions retain 87.8% of baseline accuracy
Engineering choice	PC 0-31: rank 32 retains 94.4%
Hardware measurement	Tesla T4, float32, batch 32

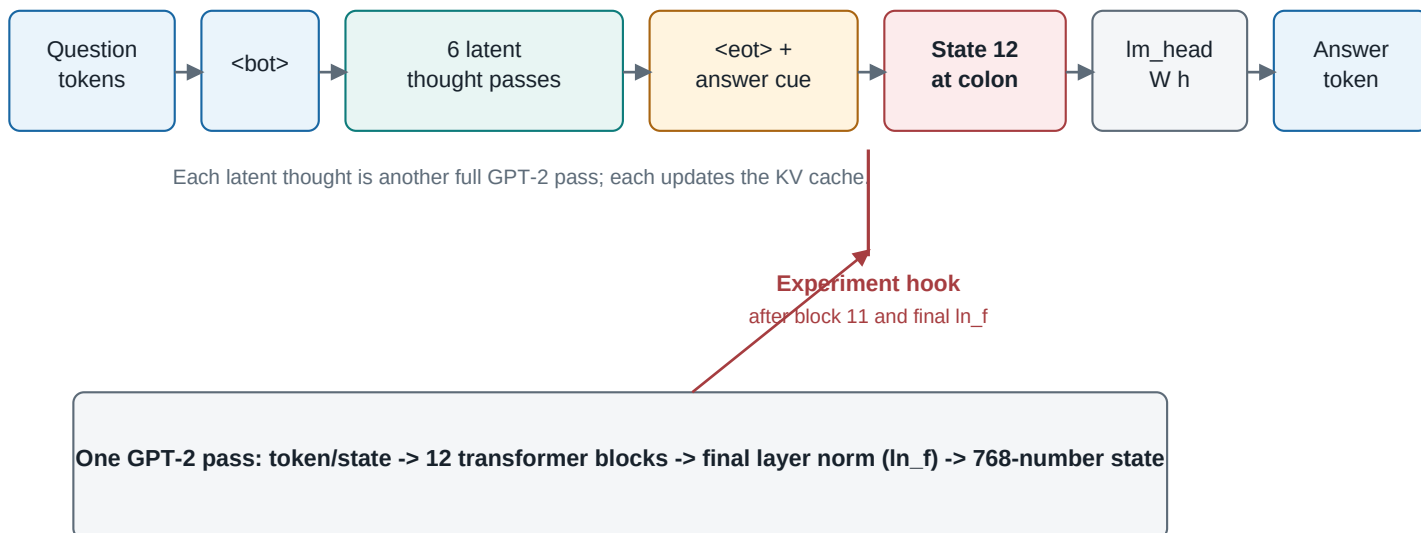
Use this as a course, not a brochure

The guide intentionally repeats the central objects - h , W , U , μ , logits - in several forms. Repetition is useful here: the experiment becomes simple only after the objects stop feeling abstract.

1. The complete story in one page

The project did not begin with a speed trick. It began with a scientific question: at the exact moment CODI is about to emit a mathematical answer, which parts of its 768-number hidden state actually matter for the answer?

CODI at inference: the exact path relevant to this experiment



State numbering used in the repository

state 0 = embedding output states 1-11 = block outputs state 12 = final normalized output

GPT-2 blocks are indexed 0-11 in code. 'State 12' is not a thirteenth transformer block.

Figure 1. CODI first performs recurrent latent computation, then reaches an answer interface. The subspace experiment acts at that interface, not inside the earlier arithmetic computation.

The experiment collected one state from each question at the colon in 'The answer is:'. A covariance matrix summarized how those states vary. PCA supplied 768 orthogonal directions ordered by variance. Causal keep-only and remove experiments then asked which groups of directions were sufficient and necessary for exact-match accuracy.

- The four largest-variance directions, PC 0-3, contain 82.3% of variance but retain only 6.1% of accuracy.
- The next 28 directions, PC 4-31, contain 11.3% of variance but retain 87.8% of accuracy.
- Keeping PC 0-31 - 32 directions total - retains 94.4% of baseline accuracy.
- That rank-32 representation makes the vocabulary readout much smaller: 768 x 50,257 becomes a 768 x 32 squeeze followed by a 32 x 50,257 score.

The key distinction: **the 28 directions are the scientific finding. Rank 32 is the engineering operating point** because adding PC 0-3 raises retained accuracy from 87.8% to 94.4% at very small additional readout cost.

2. CODI from first principles

What CODI changes about chain-of-thought

An ordinary chain-of-thought model writes intermediate reasoning as text. CODI tries to replace much of that visible text with a fixed number of continuous thoughts. A continuous thought is not a word. It is a 768-number vector produced by the model and fed back as the next input embedding.

```
question -> <bot> -> thought 1 -> thought 2 -> ... -> thought 6 -> <eot> -> 'The answer is:' -> ans
```

Each thought depends on the previous one, so these passes are sequential. The model cannot compute thought 5 before thought 4 exists. This is why removing one thought can matter for latency in a single-request setting.

Training and inference are different phases

During CODI training, a teacher path sees explicit reasoning and a student path produces silent thoughts. The checkpoint was trained to make the student's endpoint representation resemble the teacher's endpoint representation while also learning to predict the answer. Teacher and student share the same underlying model weights in the released self-distillation design.

During the experiment described here, training is already over. The official checkpoint is frozen. No weight is changed. We observe or intervene on activations during inference.

Phase	What enters the model	What this guide uses
Training	Teacher reasoning and student latent route	Background only
Normal inference	Question plus six latent thoughts	Accuracy baseline
Subspace experiment	Same frozen inference with endpoint activation editing	Find answer-bearing directions
Efficiency measurement	Same checkpoint with M sweep plus readout benchmark	Measure two optimization routes

The projection network

After each latent pass, CODI applies a learned projection module - linear, GELU, linear, then layer normalization - to turn the final hidden state into the next continuous-thought embedding. This recurrent loop is where the arithmetic workspace develops. The low-rank answer readout is later: it acts after the recurrent loop has already produced its result.

3. Inside one GPT-2 pass

The tower

Official CODI uses GPT-2 small. Its representation width is 768. A token or continuous-thought embedding travels through twelve transformer blocks. Each block contains masked self-attention, a feed-forward network, residual connections, and normalization. After block 11, a final layer normalization called `ln_f` produces the state consumed by the vocabulary head.

Repository state	Physical location	Can an edit propagate?
0	Embedding output	Yes - all twelve blocks remain
1-11	Intermediate block outputs	Yes - later blocks remain
12	Final <code>ln_f</code> output after block 11	No later transformer block; it directly enters <code>lm_head</code>

Why state 12 was useful scientifically

At state 12, the mapping to logits is exactly linear because GPT-2's output head has no bias. If h is the 768-number state and W contains one 768-number detector for each vocabulary token, then logits are simply $W h$. This lets us calculate the first-token effect of thousands of subspace interventions using matrix multiplication, after verifying parity against real generation.

```
state h (768) -> lm_head W (50,257 x 768) -> logits z (50,257) -> argmax -> token
```

Why state 12 is not the arithmetic engine

State 12 is an answer interface. It is downstream of the latent thoughts and downstream of all transformer blocks for that position. Editing it can change the scores for the next token, but it does not go backward and repair a multiplication performed in an earlier thought. This boundary explains why endpoint correction experiments failed even though the answer-bearing subspace is real.

Think of state 12 as the completed answer sheet handed to the grader. The latent thoughts are the scratch paper where the calculation happened.

4. Vectors and directions without mystery

A vector is an ordered list of numbers. In a two-dimensional drawing, a vector has two coordinates and can be drawn as an arrow. CODI's hidden state has 768 coordinates, so we cannot draw it directly, but the rules are the same.

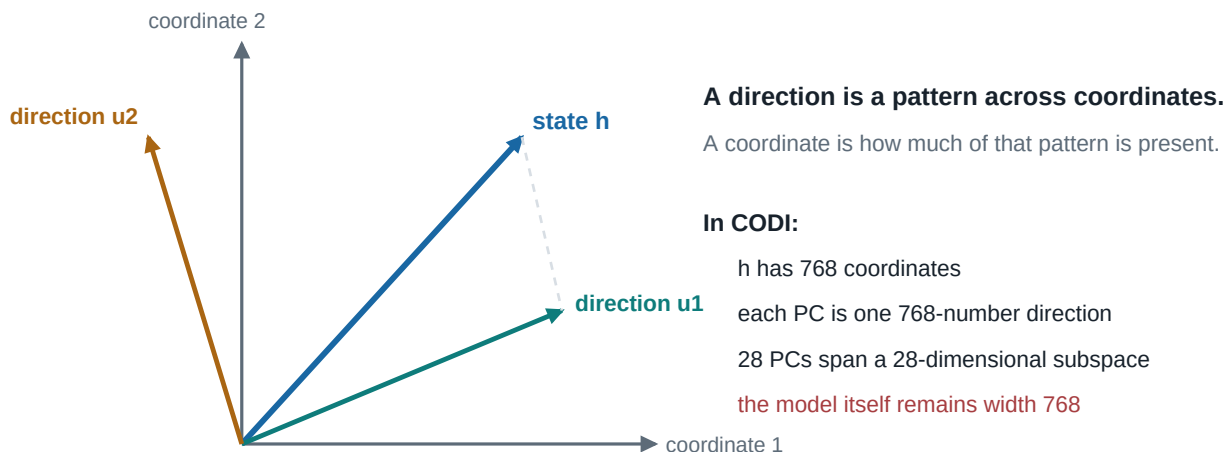


Figure 2. Coordinates are tied to the original axes; directions can be any unit-length pattern through the space. PCA finds a new set of perpendicular directions.

What one direction means here

One PCA direction is itself a list of 768 numbers. It specifies a coordinated pattern: increase some hidden coordinates, decrease others, ignore others. The amount of this direction in state h is a dot product. If u is a unit direction and mu is the average state, its coordinate is:

$$\text{coordinate} = \text{dot}(h - \mu, u)$$

A direction is not one neuron and not one column of the covariance matrix. It is a mixture of all 768 coordinates. Twenty-eight directions therefore mean twenty-eight independent patterns that together span a 28-dimensional subspace inside the original 768-dimensional space.

Basis and coordinates

A basis is a set of directions used as axes. Place 32 chosen unit directions into the columns of a matrix U. Then U has shape 768 x 32. Multiplying (h - mu) by U returns 32 coordinates. Those coordinates describe only the part of h lying inside the chosen subspace.

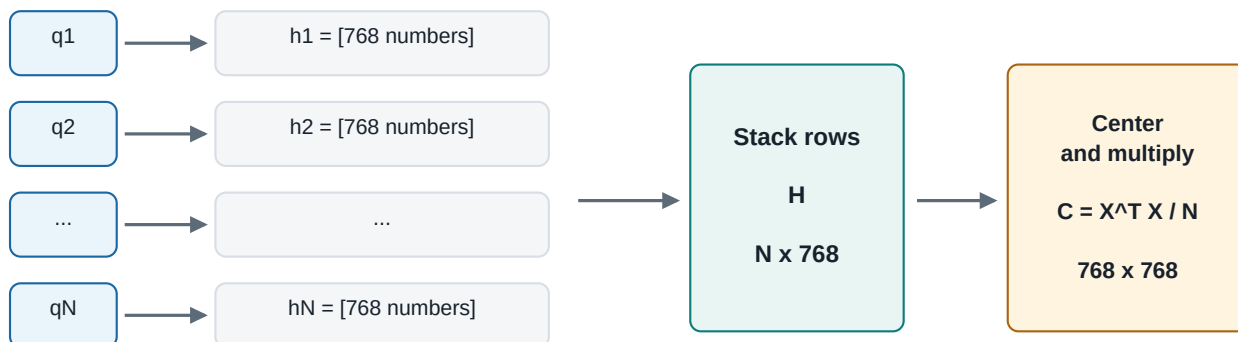
$$U = [u_0 \ u_1 \ \dots \ u_{31}] \quad c = (h - \mu) U \quad \text{shapes: } (1 \times 768)(768 \times 32) = 1 \times 32$$

5. Where the states came from

One fixed measurement location

The model was run on GSM8K questions through its released generation path. The answer cue was forced so every row reached the same semantic position. A hook captured the final $\ln_f$ output at the colon. Generation stopped after the first answer token for the large analytic sweep; full greedy decoding later confirmed the important bands on numeric exact match.

From questions to a covariance matrix



Every h is captured at the same semantic location: the colon of the forced answer cue.

Figure 3. Rows are examples; columns are hidden coordinates. The experiment never mixes different layer locations inside one covariance matrix.

Why capturing from the real generation path mattered

An earlier attempt reconstructed the endpoint with a training-style encoder and achieved only 89.06% agreement with the token emitted by generation. Differences in question normalization, cue tokenization, and left-padding widths changed absolute position IDs. The corrected collector observed the actual generation pass, raising analytic first-token parity to 1.0 in the completed run.

Data separation

The PCA basis was fit from 2,048 GSM8K-train colon states, sampled with a fixed seed and checked for zero normalized-question overlap with GSM8K test. Evaluation used all 1,319 GSM8K-test questions in fixed order. The exact-match confirmation ran the frozen checkpoint in float32 and re-established the native accuracy gate.

The direction fit is unsupervised: PCA does not use correct/wrong labels or gold answers. Gold answers enter later only to measure which retained or removed subspace preserves accuracy.

6. Covariance: a map of co-variation

Center first

Let h_i be the captured state for question i , and let μ be the average state. Centering removes the common offset:

$$x_i = h_i - \mu$$

Stack the centered states into X with one row per question. The covariance matrix is:

$$C = X^T X / N \qquad \text{shape: } 768 \times 768$$

What an entry means

Entry $C[a,b]$ is positive when hidden coordinates a and b tend to rise together across questions, negative when one rises as the other falls, and near zero when they have little linear co-movement. Diagonal entries are individual coordinate variances.

Covariance tells us	Covariance does not tell us
How endpoint states spread across questions	Which direction is correct
Which coordinate patterns co-vary	Which arithmetic step produced an error
How much variance lies along a direction	Whether that direction changes argmax
A geometry of the observed population	A causal mechanism by itself

The central trap

It is tempting to equate large variance with importance. That is not generally valid. A direction can vary enormously while adding almost the same number to every token score; then it changes confidence but not which token wins. Conversely, a modest-variance direction can rearrange the competition between specific answer tokens and be crucial for accuracy.

7. PCA: turning covariance into ordered directions

Eigenvectors and eigenvalues

PCA diagonalizes the covariance matrix. For each principal direction u_k , the covariance transformation returns the same direction scaled by an eigenvalue λ_k :

$$C u_k = \lambda_k u_k$$

The eigenvector u_k is a direction. The eigenvalue λ_k tells how much variance lies along it. Sort eigenvalues from largest to smallest and call the associated directions PC 0, PC 1, ..., PC 767.

- PC 0 is the single direction with the largest observed variance.
- PC 1 is the largest-variance direction perpendicular to PC 0.
- All PCs are orthonormal, so their coordinates do not duplicate one another.
- Together all 768 PCs can reconstruct every centered state exactly.

PCA does not create new information

PCA rotates the coordinate system. Instead of describing h using 768 original hidden coordinates, we describe it using 768 principal coordinates. Keeping only k coordinates is compression; the discarded component cannot be recovered unless additional assumptions are made.

$$h = \mu + \sum_k c_k u_k \quad \text{where } c_k = \text{dot}(h - \mu, u_k)$$

Why top-k PCA was not the final answer

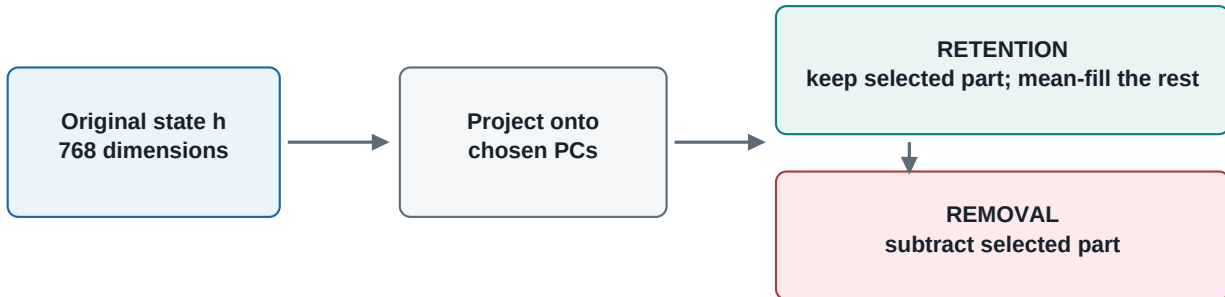
Standard PCA compression keeps PC 0 through PC $k-1$ because they explain the most variance. The experiment discovered that answer contribution is not ordered that way. The most useful answer directions form an interior band, PC 4-31. That is why causal evaluation was necessary after PCA.

PCA proposes candidate axes. Behavior tells us which axes matter for the task.

8. How the 28 directions were identified

The experiment did not simply look at eigenvalues and declare a band important. It intervened. For each candidate band it constructed edited endpoint states and measured the resulting answers.

The causal test: keep it or remove it



Sufficiency question

If only this subspace remains, does the model still answer?

Necessity question

If this subspace is deleted, does accuracy collapse?

Figure 4. Retention tests sufficiency; removal tests necessity. Using both prevents a one-sided interpretation.

Retention

Retention keeps only the selected projection around the calibration mean:

$$h_{\text{retain}} = \mu + U U^T (h - \mu)$$

If the model remains accurate, the selected subspace is sufficient for much of the answer behavior under this intervention. Everything outside the band is replaced by its average value, not literally deleted from the architecture.

Removal

Removal subtracts the selected component:

$$h_{\text{remove}} = h - U U^T (h - \mu)$$

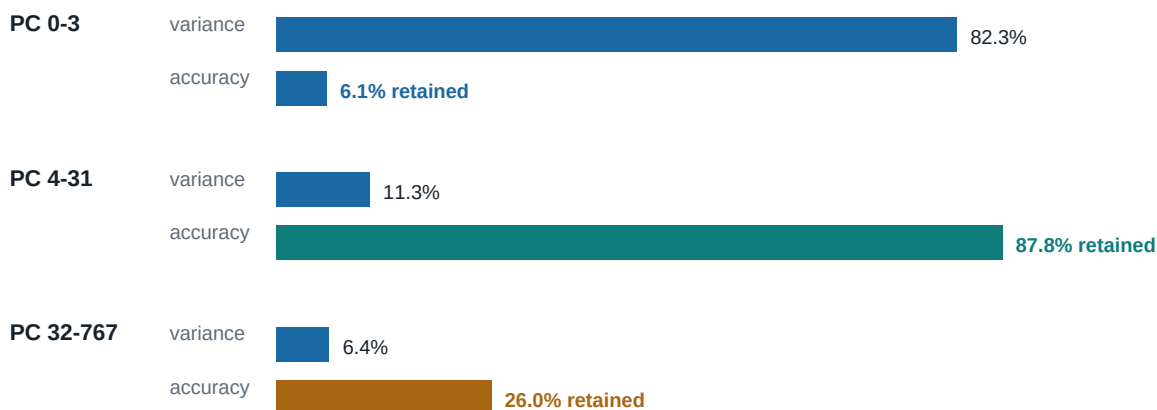
If accuracy collapses, the selected component is necessary under that edit. Random rank-matched subspaces test whether any arbitrary set of the same dimensionality would have caused comparable effects.

Why this is causal evidence, but bounded

The checkpoint and question remain fixed while the activation is changed. That supports a causal claim about this answer interface. But mean-filling can create states the model would not naturally produce, so the conclusion is about the intervention, frozen checkpoint, forced cue, and GSM8K - not every model or every possible use of the directions.

9. The result: 28 modest directions carry most of the answer

Variance is not the same as answer contribution



The answer-bearing band is not the highest-variance part of the state.

Figure 5. The accuracy figures are fractions of the unedited baseline that survive keep-only retention, not shares that must sum to 100%.

Band	Dimensions	Variance	Exact-match accuracy	Baseline retained
PC 0-3	4	82.31%	0.0265	6.1%
PC 4-15	12	7.49%	0.2191	50.5%
PC 4-31	28	11.31%	0.3806	87.8%
PC 0-31	32	93.62%	0.4094	94.4%
PC 32-767	736	6.38%	0.1130	26.0%

The forced-cue baseline exact-match accuracy was 0.4337. Removing PC 4-31 reduced it to 0.1289, a loss of 30.48 percentage points. Four random rank-28 retained subspaces achieved only 0.0281-0.0379 accuracy. All three preregistered confirmation gates passed.

The defensible sentence: at this frozen CODI answer interface, PC 4-31 are sufficient to preserve 87.8% of baseline numeric exact-match accuracy and necessary in the sense that removing them costs 30.5 points.

10. Why the top four directions do almost nothing

Logits care about differences

The predicted token is the index of the largest logit. If every logit receives nearly the same offset, their ordering barely changes:

$[2, 5, 1] + [10, 10, 10] = [12, 15, 11]$ `argmax` remains the second entry

The leading PCs behaved like this kind of common-mode lift. PC 1's readout effect shifted the vocabulary scores by about +27.1 on average with only about 0.46 spread across tokens. Large movement in state space therefore produced little movement in the identity of the winning token.

Confidence is not content

A common shift can affect quantities such as state norm or apparent confidence without encoding which number is the answer. The answer-bearing PC 4-31 band instead changes relative token scores. It acts more like a tuner choosing among stations than a volume knob raising every station together.

Property	PC 0-3	PC 4-31
Variance	Very large: 82.3%	Modest: 11.3%
Keep-only accuracy	6.1% retained	87.8% retained
Removal damage	1.52 points	30.48 points
Interpretation	Mostly common-mode/confidence geometry	Relative answer-token geometry

General lesson

Compression that keeps only the highest-variance axes would choose the wrong representation here. The safe low-rank choice was discovered by combining PCA with behavioral interventions, not by PCA alone.

11. Why 28 became 32 for engineering

This point causes most confusion because two related bases are discussed.

Name	Directions	Why it matters
Accuracy-bearing band	PC 4 through PC 31 = 28 directions	Clean scientific dissociation: modest variance, 87.8% retained accuracy
Rank-32 operating point	PC 0 through PC 31 = 32 directions	Adds four cheap coordinates and raises retained accuracy to 94.4%

The rank-32 head is not claiming that the first four PCs suddenly became answer mechanisms. It is an engineering trade: the readout cost is roughly proportional to rank, and increasing k from 28 to 32 is only a 14% increase in the compressed dimension while recovering an additional 6.6% of baseline accuracy.

rank 28: 87.8% retained accuracy, measured 120 us

rank 32: 94.4% retained accuracy, measured 117 us

rank 64: >94.4% retained accuracy, measured 154 us

The rank-28 and rank-32 timing difference is within the scale where GPU launch noise and kernel shape effects can dominate, so do not infer that rank 32 is inherently faster than rank 28. Treat both as approximately 11x in this microbenchmark.

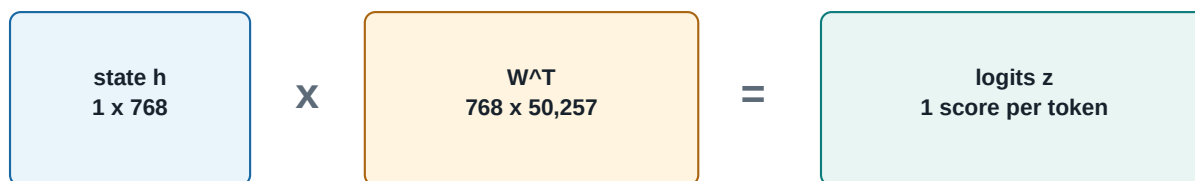
Say this aloud: 28 directions are the scientific result. Thirty-two directions are the practical low-rank head tested for a better accuracy-speed trade.

What 'keeping 32 directions' physically means

It means computing 32 dot products between the centered 768-number state and the columns of U. It does not mean deleting GPT-2 neurons, changing its hidden width, or rerunning a smaller twelve-block transformer. The saving appears only in the vocabulary-scoring stage after the state already exists.

12. The normal vocabulary readout

Normal answer readout



For token t : $z_t = \text{dot}(h, w_t)$

The highest logit wins: $\text{predicted_token} = \text{argmax}(z)$

Cost for one position: $768 \times 50,257 = 38.6$ million MACs

This is a vocabulary-scoring cost, not the cost of all twelve transformer blocks.

Figure 6. Each row of W is a token detector. All detectors are evaluated against the same hidden state.

Dimensions are your error-checking tool

Let h be written as a row vector with shape 1×768 . Let W have shape $50,257 \times 768$. Then $h W^T$ has shape $1 \times 50,257$ - exactly one score for every output token. If the dimensions do not line up this way, the equation is wrong.

Why this can be expensive

For every scored position, the model compares 768 state values against 768 weights for every vocabulary item. That is about 38.6 million multiply-accumulate operations. In rough parameter accounting, GPT-2's transformer blocks contain about 84.9 million non-embedding weights, so the full vocabulary projection is a substantial component of a word-producing pass.

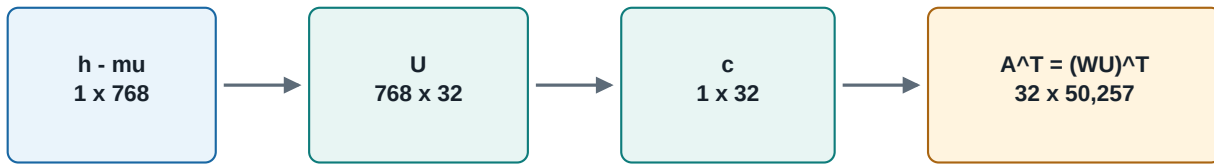
The output head is tied to embeddings

GPT-2 normally ties the output head to its token embedding table. Conceptually this still behaves as a detector matrix W : rows correspond to vocabulary items, and a dot product scores compatibility between the state and each token representation.

The low-rank readout does not help create h . It helps convert an already-created h into 50,257 token scores.

13. The low-rank algebra, one step at a time

Rank-32 readout: same projected logits, cheaper order of operations



At inference: $c = (h - \mu) U$

Projected logits: $\hat{z} = W \mu + c (WU)^T$



Important: reassociating the multiplication is exact. Discarding 736 dimensions is the approximation.

Figure 7. U is derived from the PCA basis. $A = WU$ is computed once and reused across questions.

Step 1: center and squeeze

$$c = (h - \mu) U$$

This projects the state into 32 PCA coordinates. Cost: $768 \times 32 = 24,576$ MACs per state.

Step 2: precompute the compressed token detectors

$$\begin{aligned} A &= W U & \text{shape: } 50,257 \times 32 \\ b &= W \mu & \text{shape: } 50,257 \end{aligned}$$

A and b depend on the frozen model and fitted basis, not on the current question, so they can be built offline and stored.

Step 3: score in 32 dimensions

$$\hat{z} = b + c A^T$$

The main per-state score now costs $32 \times 50,257 = 1.608$ million MACs. Including the squeeze, total arithmetic is about 1.63 million rather than 38.6 million.

Why the two forms agree

$$\begin{aligned} \text{projected state} &= \mu + ((h - \mu) U) U^T \\ W(\text{projected state}) &= W \mu + W U ((h - \mu) U)^T \end{aligned}$$

Matrix multiplication is associative, so scoring the projected 768-dimensional state and scoring its 32 coordinates against compressed detectors produce the same projected logits, up to floating-point details.

14. A toy example you can calculate by hand

Use a tiny model with hidden width $d=3$, vocabulary size $V=4$, and keep rank $k=1$. Let the chosen unit direction be $u = [1, 0, 0]^T$ and let $\mu = 0$. Then the projection keeps only the first coordinate.

$$\begin{aligned} h &= [2, 5, -1] \\ U &= [[1], [0], [0]] \\ c &= h U = [2] \\ \text{projected } h &= U c = [2, 0, 0] \end{aligned}$$

Suppose the four token detectors are rows of W :

$$W = \begin{bmatrix} 1 & 3 & 0 \\ 2 & -1 & 4 \\ -1 & 2 & 1 \\ 3 & 0 & -2 \end{bmatrix}$$

Normal scoring of the projected state gives:

$$W [2, 0, 0]^T = [2, 4, -2, 6]^T$$

Now compress the detectors once:

$$\begin{aligned} A &= W U = [1, 2, -1, 3]^T \\ A c &= [1, 2, -1, 3]^T * 2 = [2, 4, -2, 6]^T \end{aligned}$$

The answers match exactly because both computations use the same projected state. But they need not match the full-state logits $W h$, because projection discarded h 's second and third coordinates. That is the sole conceptual source of accuracy loss.

Computation	MAC count in toy
Full readout	$V d = 4 \times 3 = 12$
Rank-1 squeeze plus readout	$d k + V k = 3 + 4 = 7$

Break-even rule

The factorized cost is $k(d+V)$. It beats Vd when:

$$k < Vd / (V + d)$$

For $V=50,257$ and $d=768$, break-even is about 756. A rank of 32 is far below that, so the arithmetic reduction is large.

15. Why 24x arithmetic became 11.3x measured

Arithmetic and measured time are different



Why 24x becomes 11.3x: GPU launch overhead and memory traffic do not shrink in perfect proportion.

Figure 8. The measured bars use time normalized to the full readout for visual comparison; they are not operation counts.

Benchmark protocol

- Tesla T4, float32, batch size 32.
- Ten warm-up calls before timing.
- Two hundred repetitions.
- CUDA synchronization after each call so the clock waits for GPU completion.
- Median duration reported to reduce sensitivity to occasional stalls.

Why hardware does not scale perfectly with MACs

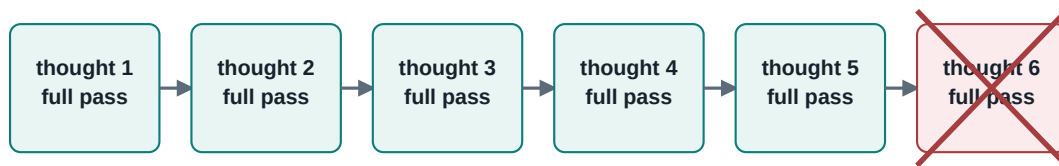
GPU matrix-vector and small matrix operations can be limited by memory bandwidth and kernel-launch overhead rather than pure arithmetic throughput. The full readout moves a much larger detector table. The compressed operation moves less data, but fixed launch and synchronization costs remain. Therefore the arithmetic ratio is about 24x while the observed component ratio is 11.3x.

A subtle implementation fact

The timing function uses a random orthonormal basis with the same dimensions as the fitted PCA basis. That is acceptable for a shape-and-kernel microbenchmark: matrix sizes and memory layout drive timing. Accuracy, however, comes from the separate real PC-band intervention. A deployment benchmark should use the actual U and include centering, bias addition, token selection, and integration into generation.

16. Lever 2: use five thoughts instead of six

Lever 2: reduce the number of sequential latent passes



M = 5 skips one recurrent pass

Measured accuracy: 43.59% at M=5 vs 43.37% at M=6

Measured batch-32 T4 wall clock: effectively flat

Estimated batch-1 architectural saving: roughly one of about sixteen sequential stages

Figure 9. This changes recurrent computation, unlike the low-rank head, which changes only vocabulary scoring.

Latent budget M	Exact match	Difference from M=6
6	0.4337	baseline trained configuration
5	0.4359	+0.23 percentage points
4	0.3836	-5.00 points
3	0.3768	-5.69 points

What this result means

M=5 showed no observed accuracy cost on this full GSM8K evaluation. The point estimate is slightly higher, but the study was a measurement sweep rather than a formal non-inferiority test. It is safest to say 'no measured loss' rather than 'M=5 is universally better.'

What it does not mean

- It does not prove that a model trained with M=5 would behave identically.
- It does not imply a smooth tradeoff; the five-point cliff at M=4 shows the opposite.
- It did not yield a measured wall-clock improvement at batch 32 on the T4.
- It is specific to this checkpoint, forced-cue decoding protocol, task, and hardware measurement.

17. Putting both levers into the full inference path

The useful accounting identity

$$\text{total time} = \text{number of sequential stages} \times \text{average cost per stage}$$

The M sweep attacks the first factor. The low-rank head attacks part of the second factor, and only on positions where vocabulary logits are needed.

Lever	Directly measured	Derived estimate
M=6 to M=5	Full-GSM8K accuracy; batch-32 wall clock flat	About one of roughly sixteen sequential stages in a latency-bound request
Rank-32 readout	1,327 us to 117 us for isolated projection	About 8-10% of a low-batch request if several stages spend about 31 us
Combined	Not measured end to end	Roughly 15% single-question latency

Latency and throughput

Latency asks how long one user waits. Throughput asks how many questions the system serves per second. At low batch size, recurrent stages and memory movement are exposed; at batch 32, GPU parallelism and fixed costs can mask the removal of one small stage. The experiment measured the latter configuration and found flat wall clock across M.

The honest headline is not 'CODI is 15% faster.' It is: **the answer-head kernel is 11.3x faster in isolation, M=5 preserves the measured accuracy, and a roughly 15% batch-1 end-to-end saving is a motivated estimate awaiting direct measurement.**

One more implementation caveat

The current efficiency script benchmarks the factorized matrix operations but does not replace `lm_head` inside `generate_official_codi`. Therefore the completed run is evidence for an optimization opportunity, not a finished production inference engine.

18. Exact versus approximate: the sentence to remember

Exact part

Once a projected state is chosen, the factorization is exact:

$$W [\mu + U U^T(h - \mu)] = W \mu + (W U) [U^T(h - \mu)]$$

Changing parentheses does not change the mathematical projected logits.

Approximate part

Replacing h by its projection discards every component outside $\text{span}(U)$. That can change logits and answers. The measured retention accuracy quantifies this approximation for the chosen population.

Statement	Correct?	Reason
The factorization approximates matrix multiplication	No	It is an algebraic reassociation
The 32-dimensional projection approximates the state	Yes	It discards 736 directions
Rank 32 preserves every answer	No	It retained 94.4% of baseline accuracy
The 28 PCs make GPT-2 width 28	No	All transformer blocks still use width 768
The low-rank head can be precomputed	Yes	W , U , and μ are frozen

A possible near-lossless extension

Use the rank-32 head to propose the top K tokens, then rescore only those K candidates with their full 768-dimensional detector rows. If the full model's winner appears in the candidate set, output is exactly recovered. This resembles speculative or draft-and-verify decoding, but it remains untested here. The essential measurement would be top- K recall of the original winner plus end-to-end latency.

19. What the experiment established - and what it did not

Established	Not established
A confirmed answer-bearing PC 4-31 band at state 12	A universal 28-dimensional reasoning mechanism
Rank 32 retains 94.4% of baseline exact-match accuracy	Lossless rank-32 generation
The isolated rank-32 readout kernel was 11.3x faster	An 11.3x faster whole model
M=5 had no observed accuracy loss on full GSM8K	A statistically proven improvement or an M=5 training result
M=4 causes a sharp accuracy cliff	A smooth adaptive-compute law
Both ideas motivate a roughly 15% batch-1 estimate	A measured 15% end-to-end latency reduction

Scientific limits inherited from the band experiment

- The basis was originally fitted on colon states from an environment whose absolute accuracy was degraded; the geometry transferred to the pinned reproducing environment, but a clean re-fit would remove that dependency.
- Retention mean-fills the complement and may move states off the natural activation manifold. A stricter donor control still retained a majority analytically, but was weaker.
- The exact claim is bounded to official CODI GPT-2, GSM8K, forced answer cue, state 12, and linear subspaces.
- The answer interface is low-dimensional; this does not imply that the preceding latent arithmetic is low-dimensional.

Engineering limits

- The current generator still uses its ordinary full `lm_head`.
- The benchmark omits a complete deployed low-rank pipeline and batch-1 timing distribution.
- No GPU-memory, energy, TTFT, inter-token latency, or P50/P95 end-to-end measurements were reported.

20. How to turn the opportunity into a completed optimization

A rigorous follow-up should modify the decoder, not merely time two matrix expressions.

Step	Implementation	Required evidence
1	Load the actual PC basis U and mean mu	Hashes match the confirmed band artifact
2	Precompute $A=WU$ and $b=W\mu$ once	Numerical parity with direct projected logits
3	Bypass full lm_head at eligible answer positions	Profiler proves the original head is absent
4	Run batch 1, 8, and 32	P50/P90 total latency, TTFT, token latency, throughput
5	Evaluate exact match and per-question token parity	Full 1,319-question paired results
6	Test top-K draft plus full candidate rescoring	Winner recall, exact parity rate, latency
7	Combine with $M=5$	Factorial comparison: full/ $M=5$ x full/rank32

The four-arm benchmark

A: $M=6$ + full readout B: $M=5$ + full readout
 C: $M=6$ + rank-32 readout D: $M=5$ + rank-32 readout

This factorial design separates the contribution of each lever and tests whether their savings compose. Each arm must use identical questions, decoding rules, precision, and warm-up. Record distributions, not only one elapsed total.

Success criteria worth freezing in advance

- Rank-32 head: at least 8x isolated speedup and accuracy within the already measured 2.4-point cost.
- $M=5$: no more than a prespecified paired accuracy loss versus $M=6$.
- Combined batch-1: positive P50 and P90 latency reduction with identical endpoint coverage.
- Top-K verification: near-perfect original-winner recall before making a lossless claim.

21. Questions your professor may ask

What exactly is one of the 28 directions?

A unit-length 768-number eigenvector of the covariance matrix fitted to centered answer-cue states. It is a coordinated activation pattern, not one neuron.

Why are they PC 4-31 instead of the top 28 PCs?

Because task contribution did not follow variance order. PCs 0-3 had enormous variance but little effect on token identity. Behavioral retention and removal isolated the interior band.

How do you know the band matters?

Keeping only PC 4-31 preserved 87.8% of baseline exact-match accuracy; removing them cost 30.48 points; matched random rank-28 subspaces retained only 6.5-8.7%.

Why use 32 directions for speed?

PC 0-31 adds the four leading PCs. Cost rises only slightly with rank, while retained accuracy rises from 87.8% to 94.4%.

Is low-rank scoring exact?

It is exact for the projected state. Projection itself is lossy relative to the full state.

Did you make the whole model 11.3x faster?

No. The isolated output projection was 11.3x faster. The transformer remains unchanged.

Did dropping one thought make inference faster?

It removes one recurrent pass architecturally, but batch-32 T4 wall clock was flat. Batch-1 latency has not been directly measured.

Why not compress using the highest-variance PCs?

That would keep PC 0-3, which retain only 6.1% of baseline accuracy. Variance is not task importance.

Does this prove reasoning is 32-dimensional?

No. It proves the final answer interface has a useful 32-dimensional approximation. Earlier latent computation can remain distributed and high-dimensional.

Where should the low-rank head be inserted?

After the final layer-normalized state exists, replacing full vocabulary scoring at eligible answer positions. It does not belong inside the transformer blocks.

22. Glossary and formula sheet

Symbol or term	Meaning
h	One 768-dimensional state at the answer-cue colon
μ	Average calibration state used for centering
H	Matrix of stacked uncentered states, one row per question
X	Centered state matrix $H - \mu$
C	768 x 768 covariance $X^T X / N$
u_k	The k-th PCA direction, a 768-number unit vector
λ_k	Variance along PCA direction u_k
U	Matrix whose columns are selected PCA directions
c	Low-dimensional coordinates $(h - \mu)U$
W	Vocabulary readout, one 768-number detector per token
z	Full vector of token scores or logits
$A = WU$	Vocabulary detectors compressed into the selected basis
$b = W\mu$	Mean-state token-score vector
MAC	Multiply-accumulate, a rough unit of arithmetic work
Latency	Time one request waits
Throughput	Requests or tokens served per unit time

Core equations

Center:	$x = h - \mu$
Covariance:	$C = X^T X / N$
PCA:	$C u_k = \lambda_k u_k$
Coordinates:	$c = (h - \mu) U$
Retention:	$h_{\text{retain}} = \mu + c U^T$
Normal logits:	$z = h W^T$
Compressed tables:	$A = W U, \quad b = W \mu$
Projected logits:	$z_{\text{hat}} = b + c A^T$
Full readout cost:	$V d$
Rank-k cost:	$d k + V k = k(d+V)$

23. Final mental model

Stage 1 - computation: CODI's recurrent latent thoughts perform the arithmetic. All transformer passes still use 768-dimensional states.

Stage 2 - answer interface: after the last block and `ln_f`, state 12 contains an answer-ready representation. PCA gives axes for this population of states.

Stage 3 - causal localization: keep/remove experiments show that PC 4-31, 28 directions, carry most of the exact-match answer behavior even though they hold little variance.

Stage 4 - engineering choice: keep PC 0-31 for a 32-dimensional answer representation that retains 94.4% of baseline accuracy.

Stage 5 - low-rank scoring: precompute WU, squeeze `h` to 32 coordinates, and score vocabulary tokens in that basis. The isolated projection is 11.3x faster on T4.

Stage 6 - honest boundary: the production decoder has not yet been replaced and batch-1 end-to-end latency has not been measured. The approximately 15% whole-request saving remains a testable estimate.

If you remember only one sentence

The experiment first discovered which directions in CODI's final answer state actually matter, then used those directions as a smaller coordinate system for vocabulary scoring; this makes the output head much cheaper without claiming that the earlier reasoning network itself became 32-dimensional.

Primary evidence

Repository ledger section 40: exact-match confirmation of the PC 4-31 band. Repository ledger section 59: latent-budget sweep and rank-k readout microbenchmark. Official checkpoint and dependency versions were pinned; raw exports were checksum-verified and reports recomputed.